OracleDAL 1 2 0 Developer Guide

Using async database access in the PLSQL Retirement solution

Audience: developers working in ContractService, CustomerService, and DispatchService. This guide explains how to use 66degrees.Data.OracleDAL 1.2.0 with the async patterns now merged into dev. Use genuine DAL async operations and await them throughout the call chain while preserving the original PL/SQL behavior.

What changed

Version 1.2.0 provides the async APIs used by the migration. Existing database-connected code on dev now uses async repository methods and associated callers. The projects were already referencing 1.2.0 before the migration; the merged change converted the application call chains rather than upgrading those package references.

Operation	API or helper used in migration
Read query results	GetTableBySQLAsync
Start a transaction	BeginTransactionAsync
Execute a write	ExecuteNonQueryAsync
Oracle date helper in write path	GetOracleCurrentDateAsync
Commit a transaction	CommitAsync

CancellationToken support was verified for the operations used by the migration. Transaction disposal was also inspected to confirm rollback of uncommitted work. This is a project usage guide, not a complete package API reference.

Where to find working examples

- ContractRepository and IContractRepository: async reads and writes.
- ContractService and IContractService: async orchestration, validation, and exception behavior.
- SetContractPropertyAsync: transactional write pattern.
- Contract, Customer, and Dispatch controllers: awaited calls and request cancellation.
- project-architecture.mdc and the service tests: current rules and test patterns.

The reported migration commit is 750b94d458c7b169592aea0365fa64f50210d566. This guide uses the migration and review results supplied by the project architect. Exact DAL overloads, namespaces, parameter objects, and transaction types must be taken from the installed package or current dev code.

Get the package and prepare your project

Start from current dev

Fetch the latest repository changes and create a feature branch using the team workflow. Preserve local work before updating your checkout. Read .cursor/rules, especially the architecture and GitLab workflow rules, before making changes.

Verify the version

ContractService, CustomerService, and DispatchService are reported to reference 1.2.0. Check the applicable project or centrally managed package reference; do not add a duplicate reference. A project-level reference has this form:

```
<PackageReference Include="66degrees.Data.OracleDAL"
                  Version="1.2.0" />
```

Run these commands from the solution root:

```
dotnet restore PLSQL-Retirement.slnx
dotnet build PLSQL-Retirement.slnx -c Release
```

Authenticate to the package feed

The internal GitLab NuGet endpoint is https://gitlab.com/api/v4/projects/85992194/packages/nuget/index.json. Use the approved developer credentials and feed configuration. Cloning through SSH does not authenticate NuGet package downloads.

A 401 from the internal feed indicates an authentication or access issue. Confirm the configured source and approved credentials before changing application code. The migration was locally verified using nuget.org and a local OracleDAL feed; that does not prove the GitLab or CI feed credentials are configured.

```
dotnet nuget list source
```

Do not commit tokens, credentials, connection strings, or local appsettings.Development.json. Keep the architect-controlled connection configuration unchanged. For Docker restores, retain the existing BuildKit NuGet configuration secret pattern.

Use the existing architecture

- Remain on .NET 8 and preserve Swashbuckle.
- Use OracleDAL; do not introduce Dapper, EF, new PL/SQL, or unapproved packages.
- Keep the established RepositoryWrapper construction pattern.
- Keep SQL in the existing infrastructure constants and use parameterized queries.
- Follow the approved IExceptionHandler strategy instead of adding redundant catches.

Make the complete call chain async

A controller awaits a service; the service awaits the repository; the repository awaits the DAL. Update interfaces and implementations together, then update every affected caller, helper, fake, and test. Renaming a method alone does not make database I/O asynchronous.

Preserve the source contract

Keep numeric precision, input defaults, nullability, output values, messages, return codes, and SQL semantics. Wrap the existing CLR result in Task<T>; use Task for a method with no result. For example, decimal becomes Task<decimal>, not Task<long>. Preserve richer result models rather than replacing them with primitive values.

Add CancellationToken as an execution-control parameter according to the existing project conventions. It is not a new PL/SQL business argument. Keep HTTP routes and calling-application literals unchanged unless a separate approved change requires it.

Illustrative interface and controller pattern

This example demonstrates async propagation. It is not the exact signature of a particular migrated method. Keep your method's existing business result type and response mapping.

```
public interface IAmountRepository
{
   Task<decimal?> GetAmountAsync(
      decimal contractId,
      CancellationToken cancellationToken);
}

public async Task<IActionResult> GetAmount(
   decimal contractId, CancellationToken cancellationToken)
{
   var amount = await _service.GetAmountAsync(
      contractId, cancellationToken);
   return Ok(amount);
}
```

In repositories and services

Select the actual 1.2.0 DAL overload in IntelliSense or inspect its metadata. Retain the existing SQL and parameter binding, await the async operation, and pass the caller's token in the position or named parameter defined by that overload. Await nested validations and lookups as well as the final query.

- Use the Async suffix consistently for repository and service methods.
- Keep dependent operations sequential; do not introduce parallel use of one connection or transaction.
- Do not use Task.Run to wrap a synchronous DAL call.
- Do not block with .Result, .Wait(), or GetAwaiter().GetResult().
- Do not use async void for application database methods.

Handle cancellation and transactions

Pass cancellation to the database

Accept the request token in controllers and pass the same token through services, repositories, helpers, and supported DAL calls. ASP.NET Core supplies the action CancellationToken from request cancellation. Do not silently substitute CancellationToken.None or create an unrelated token.

When cancellation occurs, stop subsequent business operations. OperationCanceledException includes TaskCanceledException and must propagate through the migrated service and repository paths. Do not translate it into OthersErrorException, ContractPropertyWriteException, or a normal business failure.

Keep cancellation out of error translation

Where an approved existing catch translates exceptions, retain its current exclusions and add cancellation to those exclusions. Do not introduce a new general catch merely to copy this example:

```
catch (Exception ex) when (
   ex is not OperationCanceledException
   /* retain existing exclusions here */)
{
   // Existing approved error translation only.
}
```

If an existing catch needs to explicitly preserve cancellation, use catch (OperationCanceledException) { throw; } before the other catch. Use throw; to retain the original stack. Follow the shared exception middleware conventions rather than inventing a new cancellation status response.

Use the existing transactional write as the template

SetContractPropertyAsync on dev is the reference for the transaction lifecycle. Its reported sequence is BeginTransactionAsync, GetOracleCurrentDateAsync, ExecuteNonQueryAsync, then CommitAsync. The transaction resource is managed with await using.

- Use the exact package transaction type and overloads from that implementation.
- Keep every operation in the original connection and transaction scope.
- Commit only after all required writes complete successfully.
- Preserve rollback on failures and cancellation. Uncommitted disposal was verified to roll back in 1.2.0.
- Do not add a cancelled request token to cleanup in a way that prevents rollback.
- Avoid duplicate explicit rollback and disposal logic unless required by the approved DAL pattern.

Cancellation during a write or commit is not proof that nothing persisted. Preserve the transaction cleanup behavior and do not automatically retry the whole write or report a guaranteed rollback solely because cancellation was observed.

Test and review your implementation

Update meaningful tests

Use async xUnit test methods returning Task. Await the method under test and adapt interface fakes or mocks to return the correct Task<T>. Task.FromResult is suitable for an in-memory fake; it is not a substitute for asynchronous production database access.

```
[Fact]
public async Task LookupPreservesCancellation()
{
   using var cts = new CancellationTokenSource();
   cts.Cancel();
   // Arrange the fake to throw for this token.
   await Assert.ThrowsAnyAsync<OperationCanceledException>(
      () => sut.GetAmountAsync(contractId, cts.Token));
}
```

The example assumes sut, contractId, and a configured fake are supplied by the fixture. Cover token forwarding separately; the assertion above alone does not prove that the token reached the DAL.

- Retain existing success, invalid-input, no-data, null-value, and error-mapping tests.
- Verify tokens are forwarded and cancellation is not wrapped.
- Verify dependent operations are not called after terminal results or cancellation.
- For transactional writes, verify success commits and failure/cancellation follows the rollback lifecycle.
- Preserve SQL filters, source types, status codes, messages, and route behavior.

Run the required checks

```
dotnet build PLSQL-Retirement.slnx -c Release
git diff --check
```

Run all affected test projects using their configured xUnit runner. In the migration environment, the VSTest path used by dotnet test failed with the SDK and Microsoft Testing Platform configuration, so the xUnit v3 executables were run directly. Use the actual executable paths produced by your build; do not change the SDK or add packages just to bypass the runner configuration.

Review before opening the merge request

- Check all affected domain projects are included in the solution.
- Audit production paths for remaining sync DAL calls and blocking async calls.
- Confirm resource disposal and transaction boundaries are preserved.
- Confirm no credentials, connection settings, or unrelated changes are included.
- Record the actual build command, test runner, test totals, and any environment limitations in the MR template.

Migration validation reference

Reported validation: Release solution build passed with a pre-existing CS8618 warning; ContractService passed 317 tests after the cancellation fix. DispatchService previously passed 6 tests; CustomerService had no tests in its scaffold. No live database writes or credential changes were made during migration verification. Run your own checks for each new change.